

What type of thing would you rather be able to fully forget on command so you could enjoy it again?

ex) I tell myself
"forget the Mistborn
book series", Then
I get to reread
it fresh

CS 340

Bitwise Operations 0b10

clicker.cs.illinois.edu

Q1

~Code~
340



Book
Movie
TV show
video game

Updates

1. Exam 1 retake - might take a full week to get back the rubric

2. MP3 - out ~~today~~ (due next Tuesday)

a. MP Party Today Siebel 3403

3:30-5:00

3. HW5 - ^{next} ~~due a week from~~ Wednesday

Bitwise Operations

Today's LGs - Build on your mental model of how data is stored and interpreted on a computer (1's and 0's).

Be able to converse about bits

Be able to shift bits

Be able to apply logical operations at a bit level

Be able to use bit operations to isolate the bits you need

Be able to convert between code points and UTF-8

Be able to convert between bits and negative numbers

Be able to create and use bit sets → 3.2

Agenda

1. UTF-8 *MP3.1*

2. Negative numbers

3. Bit operation practice

4. Bit sets *- MP3.2*

UTF-8

Char - 1 byte - 8 bits

ASCII - mapping from ⁰⁻¹²⁷ number → symbol
a A c

Unicode - mapping from code points → symbol

UTF-8 - encoding code points → char's



(Encoding) Code point -> UTF-8



U+2705 ← Hex

0010 0111 0000 0101

1110 0010

1001 1100 1000 0101

0xE2
0x9C
0x85

Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
<u>1110xxxx</u>	first byte of a <u>three-byte character</u>
11110xxx	first byte of a four-byte character
11111xxx	invalid

bytes

Bits needed	Groups used
0–7	1
8–11	2
<u>12–16</u>	<u>3</u>
17–21	4

(Decoding) UTF-8 -> Code point

find start
chop off headers
put together

Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
1110xxxx	first byte of a three-byte character
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups used
0–7	1
8–11	2
12–16	3
17–21	4

(Decoding) UTF-8 -> Code point

~~1110~~ 0100 ~~1000~~ 0001 ~~1001~~ 0000

0100 0000 01 00 0000 = 0x4050

映

Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
→ 1110xxxx	<u>first byte of a three-byte character</u>
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups used
0–7	1
8–11	2
12–16	3
17–21	4

MP3.1 - UTF-8

10101010...

code points $\rightarrow$ chars

chars $\rightarrow$ code points

bit operations
bit shifting

$x = 11111(010)$

$x < 5$

$x > 5$

Agenda

1. UTF-8

2. Negative numbers

3. Bit operation practice

4. Bit sets

aside overflow

finite space
infinite numbers

$$x = 0xF0$$

$$y = 0x10$$

$$x + y = 0x$$

$$x = \begin{array}{cccc} & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{array} 0000$$

$$y = \begin{array}{cccc} & & & \\ & & & \\ & & & \\ 0 & 0 & 0 & 1 \end{array} 0000$$

$$\neq \begin{array}{cccc} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{array}$$

Representing Values - 3 bit value

— — — 8 combos

000 001 010 011 100 101 110 111

unsigned

0 1 2 3 4 5 6 7

option 0 1 2 3 -0 -1 -2 -3

↓
highest
order bit
means negative

- Two 0's

- needs special hardware/logic to
do math

$$\begin{aligned} 110 \quad 001 &= 111 \\ -2 + 1 &= -3 \end{aligned}$$

2's Complement for Negatives

	000	001	010	011	100	101	110	111	
unsigned	0	1	2	3	4	5	6	7	0-7
opt 1	0	1	2	3	-0	-1	-2	-3	-3-3
opt 2	0	1	2	3	-4	-3	-2	-1	-4-3

- only 1 zero
- highest order bit still tells us it's negative

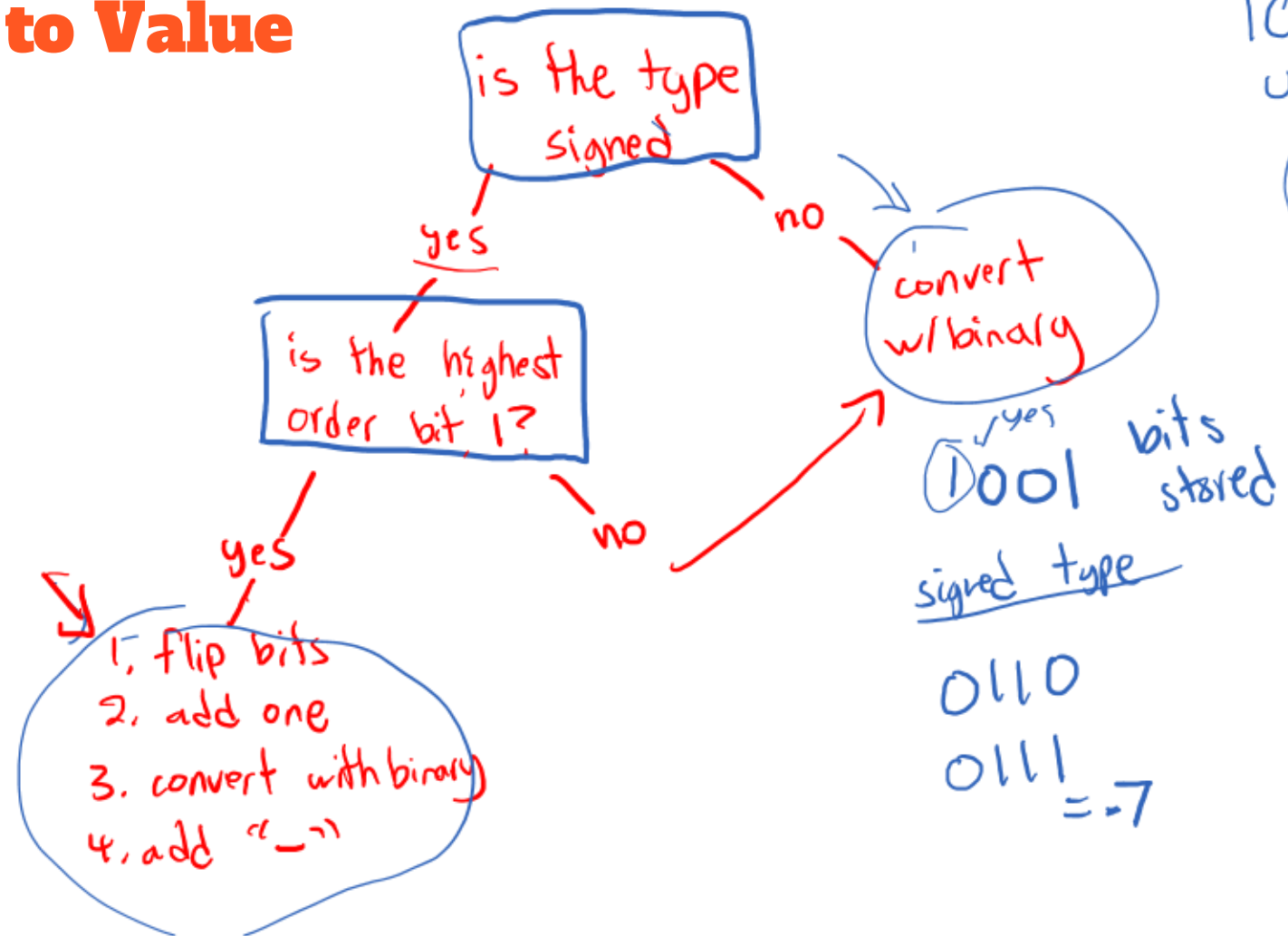
- easy to convert

- math works later

$$\begin{array}{r}
 100 \\
 \hline
 011 \text{ flip} \\
 \hline
 100 \text{ add 1}
 \end{array}$$

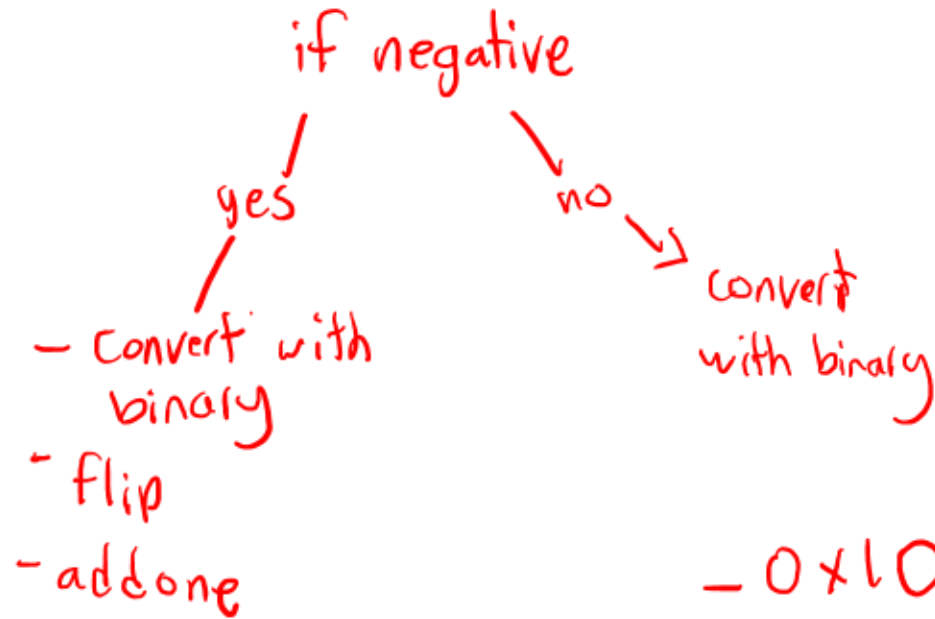
$$\begin{array}{r}
 110 \\
 \hline
 001 \text{ flip} \\
 \hline
 010 \text{ add 1} \\
 \downarrow 2
 \end{array}$$

Bits to Value



1001
unsigned
(9)

Value to Bits



- 0x10

0001 0000

1110 1111

1111 0000

Big Idea: value $\neq$ bits

$$\begin{array}{l} 0xFO \text{ — bits stored} \\ \text{1111 0000} \end{array} = \text{— } 0x10 \text{ — value the bits mean}$$

Negative Numbers with Bits

000	001 ↙	010	011	100 ↙	101 ↙	110 ↗	111 ↗
0	1	2	3	-4	-3	-2	-1

$$-2 + 1 = -1$$

$$\begin{array}{r} 110 \\ \oplus 001 \\ \hline 111 \end{array}$$

$$\begin{array}{r} 110 \quad 011 \\ + 110 \\ \hline \cancel{1} 001 \end{array}$$

-no special logic
needed for
math

-one zero

scales! 8 bits 32 bits

What value do these 8 bits represents? Assume it is a signed data type and write the answer in decimal.

0b1111 0100
negative

flip
add one
0000 1011
0000 1100 = -12

clicker.cs.illinois.edu

Q2

~Code~
340



What value do these 8 bits represents? Assume it is a signed data type and write the answer in decimal.

0b0000 1100 = 12

↑
positive

clicker.cs.illinois.edu

Q3

~Code~
340



What value do these 8 bits represents? Assume it is a unsigned data type and write the answer in decimal.

0b1000 0100 = 132

128 64 32 16 8 4 2 1

clicker.cs.illinois.edu

Q4

~Code~
340



HW Example 0b01

Making Masks with Bitwise Operations

[View](#)

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

2 bytes

```
short x = (-1) << 4;
```

Which of the following causes `x == y` to be true?

- ☐ (a) `short y = 0xf;`
- ☐ (b) `short y = - 0xf;`
- ☐ (c) `short y = 0xfff1;`
- ☐ (d) `short y = - 0x10;`

What is -1 in bits? Write the answer in binary.

0000 0000 0000 0001
1111 1111 1111 1110
1111 1111 1111 1111 +1

~0
-1
bit vector of all 1's

clicker.cs.illinois.edu

Q5

~Code~
340



Making Masks with Bitwise Operations

View

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
short x = (-1) << 4;
```

What is x in bits? Write the answer in binary.

x = 1111 1111 1111 0000

clicker.cs.illinois.edu

Q6

~Code~
340



Making Masks with Bitwise Operations

View

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
short x = (-1) << 4;
```

11111111

Does x represent a positive or negative number?

clicker.cs.illinois.edu

Q7

~Code~
340



$x = \text{1111 1111 1111 0000}$
bits stored
negative
~~what~~ is the value negative or positive

Making Masks with Bitwise Operations

View

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
short x = (-1) << 4;
```

What is x as a value (not the representation)? Write the answer in hex.

x = 0111 1111 1111 0000
0000 0000 0000 1111
- 0000 0000 0001 0000

what do these represent?

- 0x00010

clicker.cs.illinois.edu

Q8

~Code~
340



Making Masks with Bitwise Operations

View

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
short x = (-1) << 4;
```

Making Masks with Bitwise Operations

[View](#)

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
short x = (-1) << 4;
```

bits

Which of the following causes `x == y` to be true?

☐ (a) ~~`short y = 0xf;`~~

☐ (b) `short y = - 0xf;`

☐ (c) ~~`short y = 0xffff1;`~~

☒ (d) `short y = - 0x10;`

HW Example 0b02

Making Masks with Bitwise Operations

[View...](#)

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
int x = ~flip((-1) << 24);
```

Which of the following causes `x == y` to be true?

- ☐ (a) `int y = 0x1000000;`
- ☐ (b) `int y = (1 << 24) - 1;`
- ☐ (c) `int y = - 0x1000000;`
- ☐ (d) `int y = - 0xffffffff;`

What is **-1** in bits? Write the answer in hex.

0000 000 ... 0001
1111 111 ... 1110
111 ... 111

= 0xFF.FF FFFF

clicker.cs.illinois.edu

Q9

~Code~
340



Making Masks with Bitwise Operations

View... ▾

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
int x = ~((-1) << 24);
```

What is $-1 \ll 24$ in bits?
Write the answer in hex.

$0xFF000000$

$$\begin{array}{r} 32 \\ - 24 \\ \hline 8 \end{array}$$

clicker.cs.illinois.edu

Q10

~Code~
340



Making Masks with Bitwise Operations

View... ▾

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
int x = ~((-1) << 24);
```

What is x in bits? Write the answer in hex.

$x = \downarrow 00\ FF\ FF\ FF$

positive

clicker.cs.illinois.edu

Q11

~Code~
340



Making Masks with Bitwise Operations

View... ▾

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
int x = ~((-1) << 24);
```

Does `x` hold a positive or negative number?

clicker.cs.illinois.edu

Q12

~Code~
340



Making Masks with Bitwise Operations

View... ▾

Assume that `char`, `short`, and `int` are signed integer types with 8, 16, and 32 bits respectively; and that we've declared and initialized `x` as

```
int x = ~((-1) << 24);
```

What is the answer?

```
int x = ~((-1) << 24);
```

$= 0x00FF\,FF\,FF$

clicker.cs.illinois.edu

Q13

~Code~
340



Which of the following causes $x == y$ to be true?

☐ (a) `int y = 0x1000000;`

☒ (b) `int y = [(1 << 24) - 1];`

☐ (c) ~~`int y = 0x1000000;`~~

☐ (d) ~~`int y = 0xffffffff;`~~

$0x0^0FF\,FF\,FF - 1$

Bit Sets

Set - Collection of things with no repeats or enforced order

fruit baskets

apple, orange, blueberry, kiwi

in code

array of c-strings

B1 = "apple", "orange" = 13 bytes

B2 = "apple", "kiwi"

array of char

B1 = 'a', 'o' → 2 bytes

B2 = 'a', 'k'

bit set

B1 = 0000 1 1 0 0
 ↑ ↑ ↑ ↑
 apple orange du kiwi

B2 = 0000 1 0 0 1

B1 = 12 → 1 byte

B2 = 9

**How many bytes do I need
to represent a set of 16
items?**

2 bytes

↑
need 1 spot
0 or 1

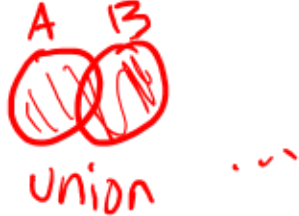
clicker.cs.illinois.edu

Q14

~Code~
340



You can use bit operations to do set manipulations!



B1 = 0x0C
B2 = 0x09

OR

$$\begin{array}{r} 0000\ 1100 \\ 0000\ 1001 \\ \hline 0000\ 1101 \end{array}$$

Union = OR

What operation would I need to find the intersection of two fruit basket sets?

clicker.cs.illinois.edu

Q15

~Code~
340



$B1 = 0x0C$

$B2 = 0x09$

goal = $0x08$ ← 0000 1000

intersection
AND

Bit Set Advantages

- space efficient
- very fast at operation, union, intersection

MP 3.2

Each word as a set of letters

find three words with no intersection

Bit Sets - Used as flags

```
int open(const char *pathname, int flags);  
int open(const char *pathname, int flags, mode_t mode);  
  
int creat(const char *pathname, mode_t mode);  
  
int openat(int dirfd, const char *pathname, int flags);  
int openat(int dirfd, const char *pathname, int flags, mode_t mode);
```

```
char path[PATH_MAX];
```

```
→ fd = open("/path/to/dir", O_TMPFILE | O_RDWR,  
             S_IRUSR | S_IWUSR);
```

$O_TMPFILE = 0x00000001 = 1$

$O_RDWR = 0x00000010 = 16$

↓ ↓
 $0x00000011 = 17$